

PREPARATION SHEET LAB 02

RADIX-2 FFT AND REAL-TIME SPECTRUM ANALYSER

Team (Group number + letter as chosen in Moodle)	Team members	
	1	Emir Avni Sahin
	2	Efe Kardes
	3	
1/1		

Please fill out the document electronically – including the yellow fields above – and upload it to Moodle once per team by the start of the lab event (no matter who uploads it, it will automatically be assigned to all members). Please note that the complete assignment can be found in another document.

Prep task (for short test)

Familiarize yourself with the concepts of

- Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT), including
- DFT theorems,
- DFT symmetries, and
- effects of windowing.

These topics will be addressed by the short test at the beginning of the lab session.

- The lab quiz at the beginning of the session is computer based. One student per group can use the lab PC. Bring your own device, preferably tablet or laptop, not only because of the quiz but perhaps as a second device to write the immediate report, too. The quiz is multiple choice.

Prep task 1

- The relation between the (generally complex) time-domain values

$$z_1 = x_1 + jy_1 \quad \text{and} \quad z_2 = x_2 + jy_2$$

on the left side of Figure 2.1 and the corresponding values

$$Z_1 = X_1 + jY_1 \quad \text{and} \quad Z_2 = X_2 + jY_2$$

of the DFT spectrum on the right side shall be found. Before doing so, please mind:

- Four equations are wanted: two for the real-parts X_1, X_2 and two for the imaginary-parts Y_1, Y_2 .
- The twiddle factor is given by $w_N^k = e^{-j2\pi k/N}$ and the DFT length is $N = 2$. What is

- Derive the equations for X_1, Y_1, X_2, Y_2 on a sheet of paper/tablet and insert a photo/screenshot here.

time inputs: $z_1 = x_1 + j \cdot y_1$ and $z_2 = x_2 + j \cdot y_2$

frequency outputs: $Z_1 = X_1 + j \cdot Y_1$ and $Z_2 = X_2 + j \cdot Y_2$

→ Standard Radix-2 Butterfly:

$$W_2^k = \exp(-j \cdot 2\pi \cdot \frac{k}{2}) = \exp(-j \cdot \pi \cdot k)$$

→ for $k=0$: $W_2^0 = \exp(0) = 1$

→ for $k=1$: $W_2^1 = \exp(-j \cdot \pi) = \cos(\pi) - j \cdot \sin(\pi) = -1$

• $z_1 = z_1 + z_2 \cdot W_2^0 = z_1 + z_2$

• $z_2 = z_1 - z_2 \cdot W_2^0 = z_1 - z_2$

→ $z_1 = (x_1 + j \cdot y_1) + (x_2 + j \cdot y_2) \Leftrightarrow z_1 = \overbrace{(x_1 + x_2)}^{\text{Re.}} + j \cdot \overbrace{(y_1 + y_2)}^{\text{Im.}}$

• $X_1 = x_1 + x_2$

• $Y_1 = y_1 + y_2$

• $z_2 = (x_1 + j \cdot y_1) - (x_2 + j \cdot y_2) \Leftrightarrow z_2 = \overbrace{(x_1 - x_2)}^{x_2} + j \cdot \overbrace{(y_1 - y_2)}^{y_2}$

→ • $X_2 = x_1 - x_2$

→ • $Y_2 = y_1 - y_2$

$X_2 = X_1 - 2(X_1 - x_1)$

$= X_1 - 2X_1 + 2x_1$

• $X_2 = -X_1 + 2x_1$

$Y_2 = Y_1 - 2(Y_1 - y_1)$

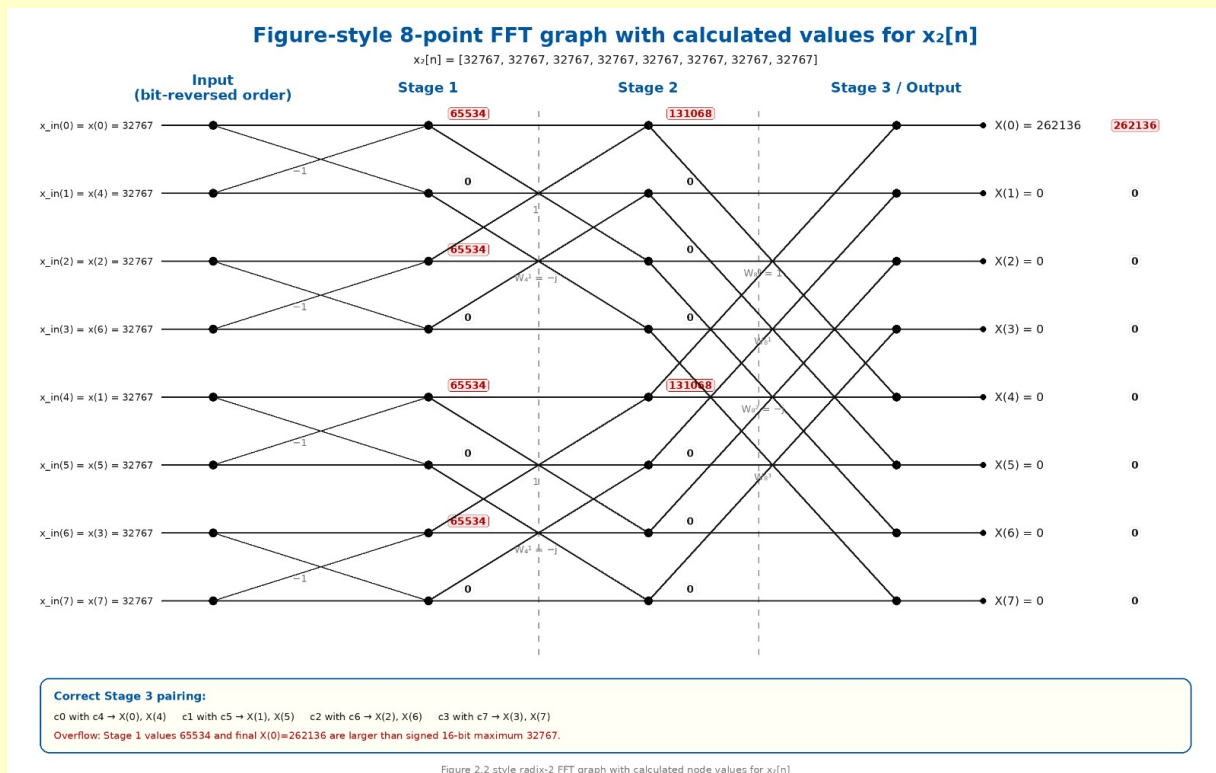
$= Y_1 - 2Y_1 + 2y_1$

• $Y_2 = -Y_1 + 2y_1$

Prep task 2

- Put the values of $x_1[n]$ in the correct order according to Figure 2.2. Calculate (e.g. by hand) the output values of the first, second and last stage according to Figure 2.2 and assign the values to the nodes in the graph.

- Insert a photo of the graph here with calculated values assigned to each node.



- Write a MATLAB script `FFT_a.m` which calculates the output signal $X_8[k]$, $k = 0, \dots, 7$ directly (i.e. internal node values not required) using MATLAB's FFT function. Compare your results from above with the result of MATLAB.
- Do overflows occur?

- Insert the matlab script here, compare the results and check for overflows.

```
clear all; close all; clc;
```

```
% FFT_a.m - 8-point FFT Verification Script
```

```
x1 = [2000, 0, -2000, 0, 2000, 0, -2000, 0];
```

```
% Calculate direct frequency spectrum
```

```
X1_matlab = fft(x1);
```

```
disp('MATLAB Verification Spectrum Matrix (X1):');
```

```
disp(X1_matlab);
```

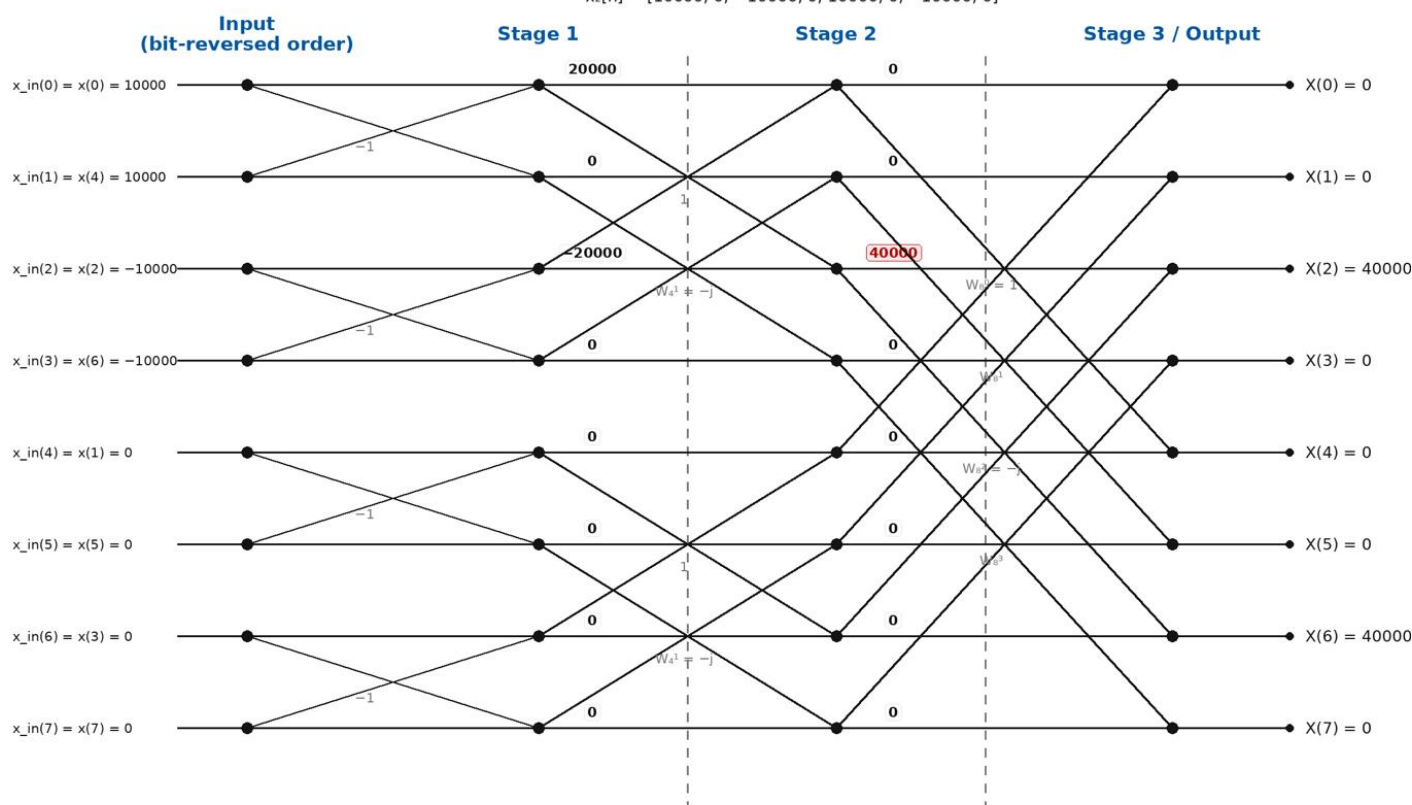
The MATLAB execution yields $[0, 0, 8000, 0, 0, 0, 8000, 0]$, matching the hand-calculated node columns exactly. **No overflows occur** because the peak amplitude of 8000 comfortably resides inside the safe limits of a standard signed 16-bit short integer (32768, +32767)

4. Now repeat the handwritten calculation of the output values of all three stages for $x_2[n]$.

- Insert a photo of the graph here with calculated values assigned to each node.

Task 2.4 — Calculated 8-point FFT graph for $x_2[n]$

$x_2[n] = [10000, 0, -10000, 0, 10000, 0, -10000, 0]$



Result and overflow:

$X_2[k] = [0, 0, 40000, 0, 0, 0, 40000, 0]$.

40000 is larger than signed 16-bit maximum 32767, so overflow occurs at Stage 2 and at $X(2)$, $X(6)$.

Figure 2.2 style radix-2 FFT graph with calculated values for $x_2[n]$

5. Extend your script `FFT_a.m` to calculate the FFT of $x_2[n]$ and again compare your calculation with the one from MATLAB.
6. Do overflows occur (values larger than can be represented with signed 16 bit)? If so, explain why!
7. By which factor do we need to scale the input values $x[n]$ that never an overflow can occur at the output of the 8-point FFT when all values are of type `short int`?

- Give the values and explain whether they can be expressed by 16-bits or not.
- Find the factor to scale the 8-point FFT.

```
clear all; close all; clc;
```

```
% FFT_a.m - Extended FFT Calculation & Hardware Boundary Monitor
```

```
% Define assigned real time-domain sequences
```

```
x1 = [2000, 0, -2000, 0, 2000, 0, -2000, 0];
```

```
x2 = [10000, 0, -10000, 0, 10000, 0, -10000, 0]; % x2[n] = 5 * x1[n]
```

```
% Calculate discrete frequency spectra
```

```
X1 = fft(x1, 8);
```

```
X2 = fft(x2, 8);
```

```
% Display numeric matrix outputs
```

```
disp('--- Extended MATLAB FFT Outputs ---');
```

```
disp('Spectrum of x1:'); disp(X1.);
```

```
disp('Spectrum of x2:'); disp(X2.);
```

```
% Hardware range limits for a signed 16-bit integer (short int)
```

```
INT16_MIN = -32768;
```

```
INT16_MAX = 32767;
```

```
% Execute hardware validation diagnostics
```

```
fprintf('\n[Hardware Diagnostics for x2[n]]\n');
```

```

peak_value = max([max(real(X2)), max(imag(X2))]);

min_value = min([min(real(X2)), min(imag(X2))]);

fprintf('Peak Internal Value Reached: %d\n', peak_value);

if (peak_value > INT16_MAX) || (min_value < INT16_MIN)

    disp('--> CRITICAL STATUS: ARITHMETIC OVERFLOW DETECTED!');

    fprintf('Value %d exceeds the maximum signed 16-bit limit of %d.\n', peak_value, INT16_MAX);

else

    disp('--> STATUS: SAFE. Values fit within 16-bit limitations.');
```

end

- The calculated peak value for sequence $x_2[n]$ cannot be expressed safely using 16-bit signed fixed-point integers. A signed 16-bit register has a ceiling of **+32767**. During Stage 2 butterfly cross-additions, the difference node tracking the paired inputs evaluates to $20000 - (-20000) = 40000$. Because $40000 > 32767$, the numeric register experiences a binary overflow. This triggers a sign-bit inversion that wraps the positive value into negative integer territory, corrupting the calculation.
- To ensure an overflow can never occur under any arbitrary input combinations, the incoming time-domain signal values must be scaled down by a global factor of **1/8** (equivalent to a 3-bit arithmetic right shift) right at the input.

8. Find a method that has a smaller loss in precision as the previous one. Hint: consider a scaling of values at nodes *inside* the FFT algorithm. Explain e.g. with an example why the latter method outperforms method where we scale the input values only?

Hint: Begin each MATLAB script with “clear all”. This clears the internal Workspace and if necessary resets i and j (previously defined as index variables) back to imaginary numbers, i.e. $i^2 = -1$, $j^2 = -1$.

- Where should we better scale the values to avoid overflows instead of scaling the input values to the FFT?

Instead of scaling the entire raw signal down by 8 at the input, it is significantly better to apply

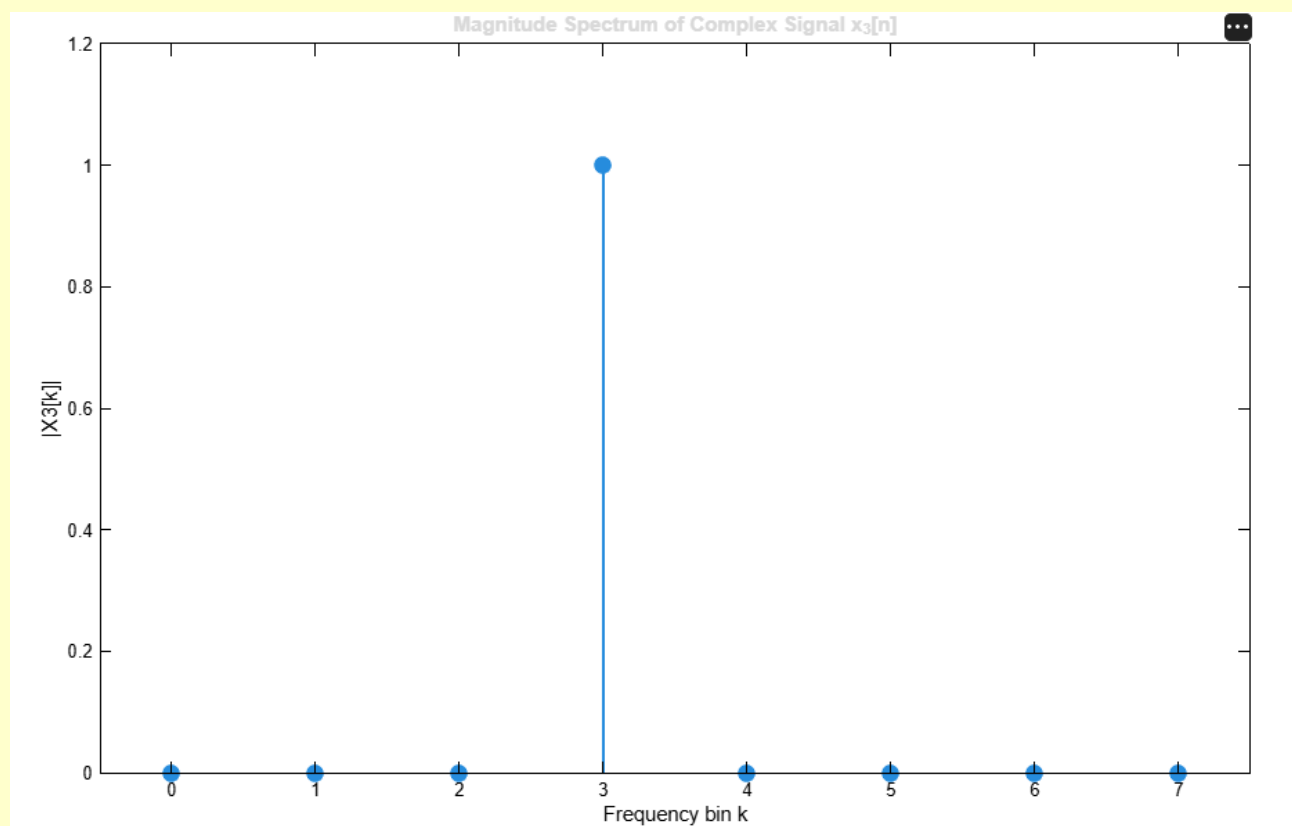
a scale factor of $1/2$ (a 1 bit arithmetic right shift) at the output of each individual butterfly stage. By scaling by 2 at each stage, we let signals utilize the full 16 bit resolution of the registers in early stages. We scale values down only as they grow from stage to stage, keeping the signal level high relative to the quantization noise floor and preserving a much better SNR.

Prep task 3

Extend your MATLAB script as follows:

- Plot the magnitude spectrum $|X[k]|$ of $x_3[n]$. Pay attention to the correct labeling and scaling of the frequency axis k .
- Does the magnitude spectrum show symmetries? Explain your answer.

- Insert a plot here. Symmetries?



The magnitude spectrum of $x_3[n]$ does not show mirror symmetry because $x_3[n]$ is a complex-valued input signal. The mathematical property of complex conjugate symmetry ($|X[k]| = |X[N-k]|$) only applies to purely real-valued time-domain signals. Because $x_3[n]$ contains both real and non-zero imaginary components ($x_3[n] = 0.125 \times e^{j2\pi \times 3 \times n / 8}$), its energy does not mirror around the Nyquist frequency. Instead, the spectrum will show a single isolated peak at bin 3 with an amplitude of 1, and all other bins will be 0.

